TECHNICAL TRAINING · ORIENTATION

Architectural Patterns for Distributed Systems

The recurring designs behind systems that scale, fail politely and can be explained to the next engineer.

Vivekanand Pujar | @VivekanandPujar

Three- or five-day workshop · Architects and senior engineers

Eight sessions · Patterns for the node, the request path and the event stream, with six case studies

SESSION MAP

Agenda

01 **Why patterns**

Three families, and the question each one answers.

02 **Single-node patterns**

Init containers, sidecars, ambassadors and the anti-corruption layer.

03 **Aggregation patterns**

The API gateway, the backend for frontend and log aggregation.

04 **Load balancing**

Layers 3, 4 and 7, and balancing from the client.

05 **Case studies: the request path**

Kubernetes Services, Fluentd, Redis and the AWS ALB.

06 **Communication modalities**

Synchronous and asynchronous: REST, gRPC, AMQP, Kafka and Pub/Sub.

07 **Event-based patterns**

Notification, state transfer, event sourcing and CQRS.

08 **Case studies: the event path**

RabbitMQ, Kafka, and Kafka as an event store.

THE ARGUMENT

What this training sets out to establish

> Distributed systems fail in a small number of familiar ways. Patterns are the agreed responses, and a team that shares them argues about the design rather than the vocabulary.

Put each concern where it belongs

Sidecars, ambassadors and gateways lift cross-cutting work out of business code and into one tested place.

Choose the coupling deliberately

Calls, notifications and carried state bind services in different ways. The choice is architectural, not incidental.

Every pattern sends an invoice

Each trades one problem for another: a hop, a library, a stale copy. We teach the bill alongside the benefit.

Vivekanand Pujar | @VivekanandPujar

SESSION 01 · WHY PATTERNS

Three families of pattern, and the question each one answers

SINGLE-NODE

What should travel with the application?

Containers that share a pod: the init container, the sidecar and the ambassador, plus the anti-corruption layer at the border with legacy.

AGGREGATION

How do many things appear as one?

Callers, services and instances gathered behind a single point: the gateway, the backend for frontend, log aggregation and load balancers.

EVENT-BASED

What travels in the message?

Services that talk without waiting: notification, state transfer, event sourcing and CQRS.

> **The recurring question.** For every pattern we ask three things: what it moves, what it couples, and what it costs. A pattern with no cost is a sales pitch.

Vivekanand Pujar | @VivekanandPujar

SESSION 02 · SINGLE-NODE PATTERNS

Four patterns keep the application small by moving everything else next door

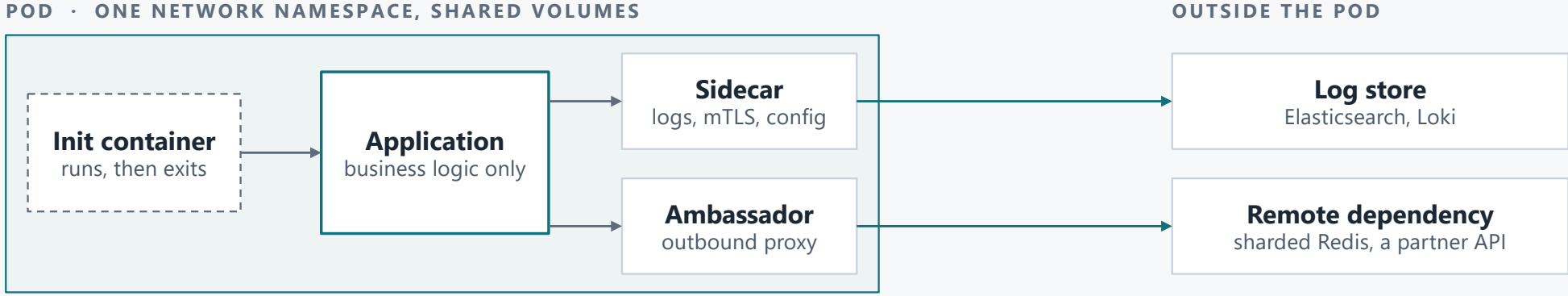
PATTERN	WHAT IT DOES	WHEN TO REACH FOR IT	FAMILIAR EXAMPLE
Init container	Runs to completion before the application starts, then exits.	Preconditions: a schema migration, waiting for a dependency, fetching secrets.	A database migration that must finish before the API takes traffic.
Sidecar	Runs beside the application for its whole life, sharing its network and storage.	Cross-cutting capability that should not live in every codebase: proxying, log shipping, config reload.	Envoy in a service mesh; a log shipper such as Fluent Bit.
Ambassador	A sidecar that speaks outward on the application's behalf.	Retries, timeouts or sharding for a remote dependency, with no change to the client.	Twemproxy or Envoy in front of a sharded Redis.
Anti-corruption layer	A translation layer between two models, so that one cannot contaminate the other.	Integrating a legacy or third-party system whose model you would rather not adopt.	An adapter that turns fixed-width legacy records into clean domain objects.

> **Three patterns decide where code runs.** The fourth decides whose vocabulary wins.

Vivekanand Pujar | @VivekanandPujar

SESSION 02 · SINGLE-NODE PATTERNS

The init container goes first and leaves; sidecar and ambassador stay to the end



Init container: ordering

Guarantees an order that Kubernetes would not: the application never starts against an unmigrated schema.

Sidecar: a shared life

Shares the pod's network and volumes, so it can proxy, ship logs or reload configuration. Cost: a proxy per pod.

Ambassador: a diplomat

The application talks to localhost; the ambassador deals with the world, including retries, timeouts and sharding.

Since Kubernetes 1.33, native sidecar containers are stable: they start before the application and stop after it.

Vivekanand Pujar | @VivekanandPujar

SESSION 02 · SINGLE-NODE PATTERNS

An anti-corruption layer stops a legacy model's quirks from spreading



ONE RECORD, TRANSLATED

LEGACY FIELD	LEGACY VALUE	DOMAIN MODEL
CUST_TYP	"B"	CustomerType.BUSINESS
INV_STAT	"09"	InvoiceStatus.OVERDUE
AMT	"000012550"	Money(125.50, EUR)

WHAT THE LAYER DOES

Translates names, codes and units into the domain's own.

Validates at the border, so bad records stop there.

Absorbs change: when the legacy format moves, only the layer does.

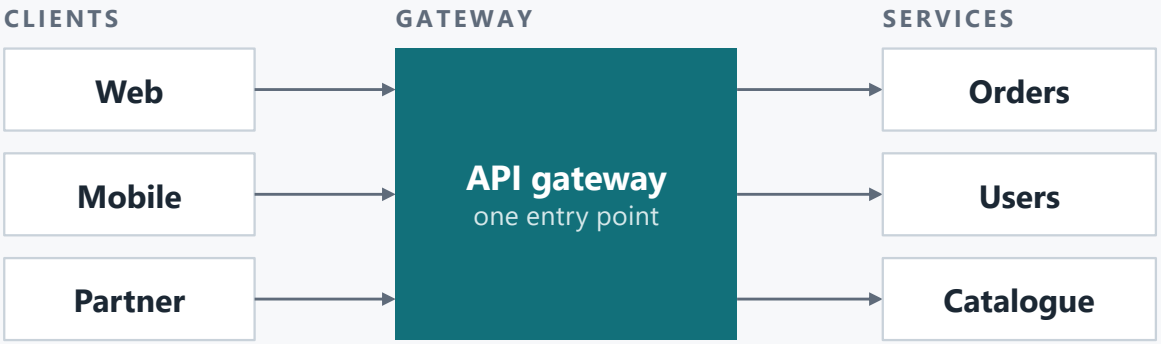
> **The test.** If a legacy status code appears in your domain code, the layer has leaked.

Illustrative scenario. The layer is the usual companion of a strangler-fig migration: as legacy functions are replaced, it shrinks.

Vivekanand Pujar | @VivekanandPujar

SESSION 03 · AGGREGATION PATTERNS

An API gateway is a single front door: keep it a door, not a department



In the wild. Amazon API Gateway, Kong, Envoy Gateway, Spring Cloud Gateway.

The toll. It sits on the request path, so every call pays one more hop.

WHAT IT TAKES OFF EVERY SERVICE

Routing. Public paths map to internal services.

Authentication. Tokens and TLS are handled once, at the edge.

Rate limits. One greedy client cannot starve the rest.

Aggregation. One call fans out and returns one answer.

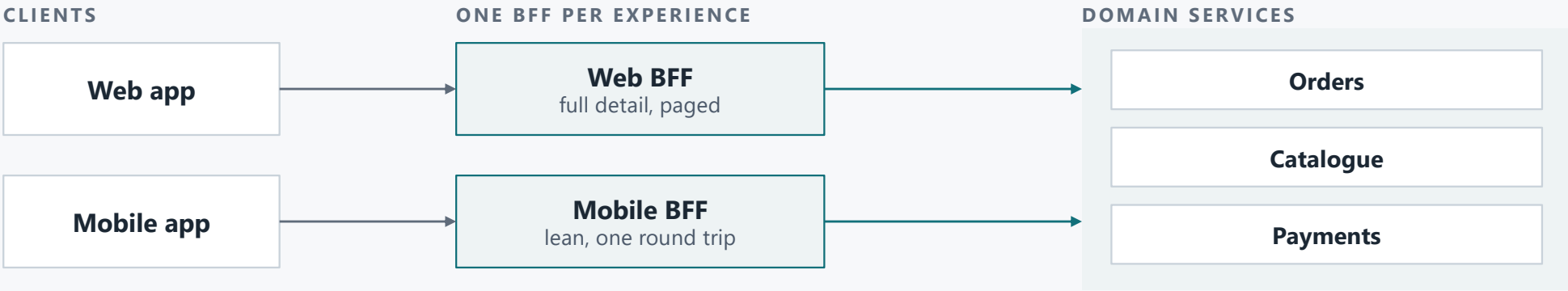
Observability. Access logs and metrics for every request.

> **The trap.** Business logic creeps in until the gateway is a distributed monolith with one team's release calendar. Keep it to cross-cutting concerns.

Vivekanand Pujar | @VivekanandPujar

SESSION 03 · AGGREGATION PATTERNS

A backend for frontend gives each client the API it would have designed



Web BFF: rich

Returns the whole order with its history and 40 fields, paged, for a wide screen.

Mobile BFF: lean

Returns a summary of six fields and resized images in one round trip, for a patchy connection.

The cost: duplication

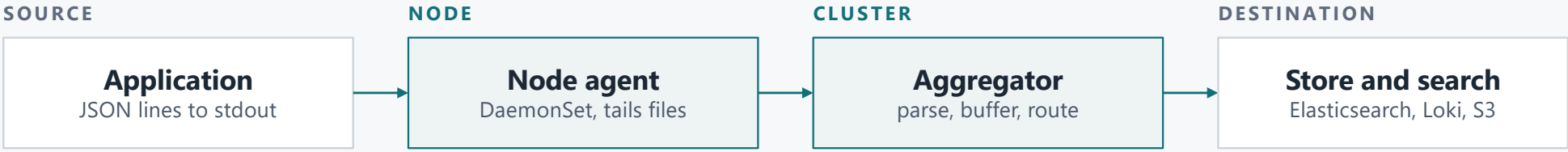
Each BFF repeats some aggregation. Share it through libraries, or push it down into the services.

> **The rule.** The team that builds the client owns its BFF, so a screen change never waits in another team's queue.

Vivekanand Pujar | @VivekanandPujar

SESSION 03 · AGGREGATION PATTERNS

Pods are disposable, so their logs must leave before they do



Write to standard output

The application emits one JSON object per line and neither knows nor cares where logs go.

Carry a correlation ID

Every line holds the request or trace ID, so one request can be followed across a dozen services.

Buffer at both ends

Agents and aggregators buffer to disk. A backend outage then delays the logs instead of losing them.

ONE STRUCTURED LINE

```
{"ts":"2026-10-02T09:14:07Z","level":"ERROR","service":"payments","order":"A-1042",
"trace_id":"4bf92f3577b34da6a3ce929d0e0e4736","msg":"card declined"}
```

Vivekanand Pujar | @VivekanandPujar

SESSION 04 · LOAD BALANCING

The higher the layer, the more the balancer understands and the more each request costs

LAYER	IT SEES	IT DECIDES ON	USE IT FOR	EXAMPLES
L3 NETWORK	IP packets	Destination address; routing and hashing, as in ECMP and anycast.	Spreading traffic across sites, or through inspection appliances.	Anycast DNS; AWS Gateway Load Balancer
L4 TRANSPORT	TCP and UDP connections	Address and port; each connection stays on one target.	Raw throughput, non-HTTP protocols, static IP addresses.	AWS Network Load Balancer; kube-proxy; IPVS
L7 APPLICATION	HTTP requests, headers and paths	Host, path, header, cookie or method.	Content-based routing, TLS termination, canary releases.	AWS ALB; NGINX; Envoy

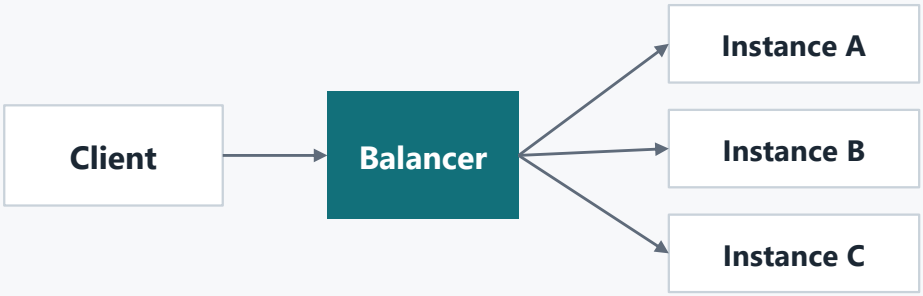
> **The rule of thumb.** Use the lowest layer that can make the decision you need. Layer 7 reads the request, but only by ending the connection first.

Vivekanand Pujar | @VivekanandPujar

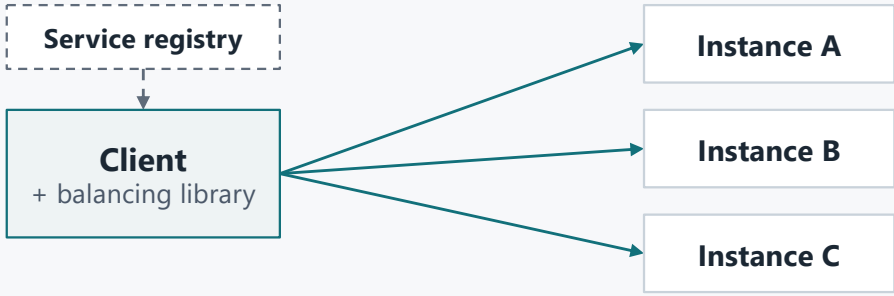
SESSION 04 · LOAD BALANCING

Client-side balancing removes a hop and adds a dependency to every client

SERVER-SIDE · A BALANCER IN THE MIDDLE



CLIENT-SIDE · THE CLIENT CHOOSES



What it buys

One hop fewer, no central bottleneck, and the client can choose on real signals such as latency and requests in flight.

What it costs

A library in every language, each needing the same upgrade, and a registry whose stale entries send traffic to dead instances.

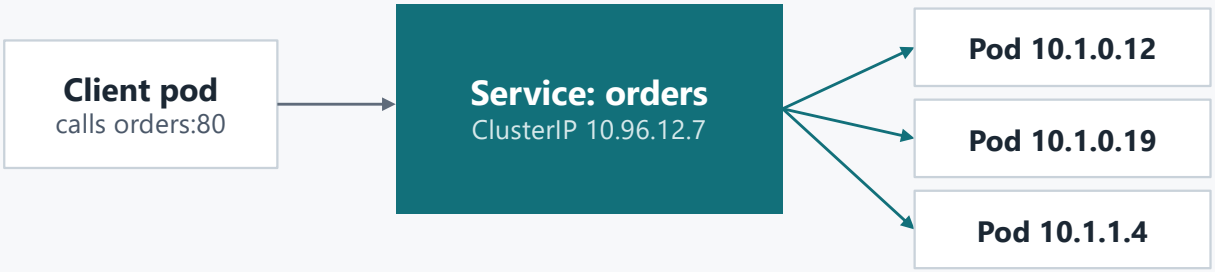
> **In practice.** gRPC, Spring Cloud LoadBalancer and Envoy balance at the client, often by “power of two choices”: pick two instances at random, send to the less busy.

Vivekanand Pujar | @VivekanandPujar

SESSION 05 · CASE STUDY 1: KUBERNETES SERVICES

A Kubernetes Service is a stable name in front of unstable pods

ONE SERVICE, THREE PODS



No process in the data path. kube-proxy on every node writes iptables or IPVS rules that rewrite the virtual IP to one healthy pod.

Readiness decides membership. A pod that fails its readiness probe leaves the EndpointSlice, and so the rotation, within seconds.

> **In practice.** A rolling update replaces all three pods. Clients keep calling orders:80 and notice nothing.

SERVICE TYPES

ClusterIP. A stable virtual IP and DNS name, reachable inside the cluster.

NodePort, LoadBalancer. Expose the Service outside; the second asks the cloud for a balancer.

Headless. No virtual IP: DNS returns pod addresses, for clients that balance themselves.

Layer 4 only. No path or header routing. For that, add an Ingress or Gateway.

Vivekanand Pujar | @VivekanandPujar

SESSION 05 · CASE STUDY 2: FLUENTD

Fluentd collects on every node, enriches with context and buffers against failure

```
<source>
  @type tail
  path /var/log/containers/*.log
  tag kube.*
  <parse>
    @type json
  </parse>
</source>
<filter kube.**>
  @type kubernetes_metadata
</filter>
<match kube.**>
  @type elasticsearch
  host logs.internal
  <buffer>
    @type file
    path /var/lib/fluentd/buffer
    retry_max_times 12
  </buffer>
</match>
```

FOUR STAGES, ONE PIPELINE

Input. Tails every container log file on the node.

Filter. Adds pod, namespace and labels from the Kubernetes API.

Buffer. Writes chunks to disk and retries with back-off.

Output. Ships to Elasticsearch, S3 or Kafka.

Two tiers. Fluent Bit on each node, Fluentd as the aggregator.

Illustrative configuration, abbreviated. Fluentd is a CNCF graduated project; Fluent Bit is its lighter sibling.

SESSION 05 · CASE STUDY 3: REDIS

Redis Cluster puts the routing table in the client: 16,384 slots and no middleman

THE ROUTING RULE

```
slot = CRC16("user:1042") mod 16384 = 10999  → Node C
```



No middleman

Clients talk to the owning node directly: no proxy hop, and no proxy to scale.

Redirection on change

A stale map earns a MOVED reply and the client updates it. ASK covers a slot mid-migration.

The price

Smart clients, and multi-key commands only within one slot. Hash tags such as {user:1042} keep keys together.

The alternative is an ambassador. Twemproxy or Envoy keeps clients simple, at the price of one more hop.

Vivekanand Pujar | @VivekanandPujar

SESSION 05 · CASE STUDY 4: AWS APPLICATION LOAD BALANCER

An Application Load Balancer reads the request and routes by what it says



PRIORITY	IF THE REQUEST…	THEN THE ALB…
1	has a path under /orders/*	forwards 90 % to orders-v1 and 10 % to orders-v2, a canary release
2	arrives for host admin.example.com	authenticates the user with OIDC, then forwards to admin
Default	matches nothing above	forwards to web

> **Choose well.** Reach for the ALB when the route depends on the request, and for the NLB when it does not.

Vivekanand Pujar | @VivekanandPujar

SESSION 06 · COMMUNICATION MODALITIES

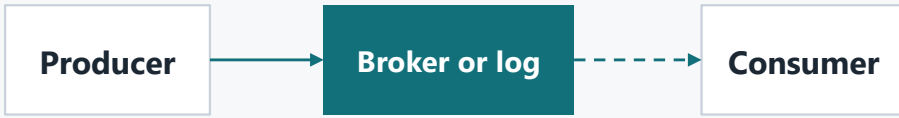
Synchronous calls couple services in time; asynchronous messages decouple them

SYNCHRONOUS · THE CALLER WAITS



caller blocked until the reply

ASYNCHRONOUS · THE CALLER MOVES ON



producer carries on; consumer reads when ready

SYNCHRONOUS

REST

Resources over HTTP. The lingua franca of public APIs.

gRPC

Typed contracts over HTTP/2, with streaming.

ASYNCHRONOUS

AMQP

Broker-routed messages with acknowledgements.

Kafka

A partitioned, replayable log of events.

Pub/Sub

Managed topics with fan-out to subscribers.

> **The coupling test.** If the caller must fail when the receiver is down for ten minutes, the coupling is synchronous, whatever the diagram says.

Vivekanand Pujar | @VivekanandPujar

SESSION 06 · COMMUNICATION MODALITIES

Each modality suits one kind of conversation and bills for it differently

MODALITY	BEST FOR	STRENGTHS	TRADEOFFS
REST	Public APIs, resource CRUD, browser and partner clients.	Universal tooling, HTTP caching, easy to debug.	Verbose payloads; no built-in streaming or strict contract.
gRPC	Internal calls between services, streaming, polyglot teams.	Typed Protobuf contracts, compact, HTTP/2 multiplexing.	Awkward in browsers; harder to inspect; long-lived connections defeat L4 balancing.
AMQP	Task queues, work distribution, routed commands.	Acknowledgements, flexible routing, per-message TTL.	Messages vanish once consumed; ordering weakens with competing consumers.
Kafka	Event streams, many independent readers, replay.	Durable log, high throughput, ordering per key.	Operational weight; consumers manage offsets; no per-message routing.
Pub/Sub	Managed fan-out to many subscribers.	Nothing to run; scales on demand.	Provider lock-in; ordering and replay are opt-in, with limits.

AMQP here means RabbitMQ. Pub/Sub covers Google Cloud Pub/Sub and, in spirit, Amazon SNS.

Vivekanand Pujar | @VivekanandPujar

SESSION 06 · COMMUNICATION MODALITIES

Six misuses turn sound protocols into distributed headaches

Chained calls

A calls B calls C calls D. Four services at 99.9% each give 99.6% together.

Instead: Cut the chain with an asynchronous hand-off.

Retry storms

Three layers retry three times each, and one slow database receives 27 times the load.

Instead: Retry at one layer, with back-off, jitter and a budget.

Chatty interfaces

Fifty fine-grained calls to draw one screen, each paying full latency.

Instead: Aggregate in a BFF, or batch and stream.

RPC in disguise

Publish a request, then block on a reply queue: all the coupling, none of the simplicity.

Instead: Call directly, or let the caller carry on.

Wrong shape

Jobs on a log, where one poison message blocks its partition; history on a queue, which forgets.

Instead: Jobs to a queue, facts to a log.

Trusting exactly-once

Delivery is at least once, so duplicates arrive and a card is charged twice.

Instead: Make consumers idempotent; key every operation.

> **The common root.** Each misuse hides a cost the box-and-arrow diagram never draws: time, load or duplicates.

Vivekanand Pujar | @VivekanandPujar

SESSION 06 · COMMUNICATION MODALITIES

The matrix: let your two hardest requirements choose the modality

REQUIREMENT	REST	gRPC	AMQP	KAFKA	PUB/SUB
Needs an immediate answer	●●●	●●●	●○○	○○○	○○○
Survives a receiver outage	○○○	○○○	●●●	●●●	●●●
Fan-out to many consumers	○○○	○○○	●●○	●●●	●●●
Replay of history	○○○	○○○	●○○	●●●	●●○
Very high throughput	●○○	●●○	●●○	●●●	●●●
Browser and partner clients	●●●	●○○	●○○	○○○	●○○
Strict typed contract	●○○	●●●	●○○	●●○	●●○
Operational simplicity	●●●	●●○	●●○	●○○	●●●

●●● strong fit ●●○ good ●○○ possible with effort ○○○ poor fit

> **Reading it.** More dots mean a better fit, not a better technology. Circle your two hardest rows first.

Vivekanand Pujar | @VivekanandPujar

SESSION 06 · COMMUNICATION MODALITIES

Choose by the failure you can tolerate, not by the protocol you prefer

RULES OF THUMB

Queries are calls; facts are messages. Ask synchronously, announce asynchronously.

Must succeed now? Call. May finish later? Queue it. A queue buys time; a call buys certainty.

Across a boundary you do not own, use REST. Inside one you do, gRPC.

One consumer per job: a queue. Many readers per fact: a log or a topic.

No call without a timeout. An unbounded wait is an outage in waiting.

Assume duplicates, delays and reordering. Then design the handler so they do not matter.

CHECKLIST BEFORE YOU COMMIT

- ☐ What happens if the receiver is down for ten minutes?
- ☐ Does the caller need the answer to continue?
- ☐ How many consumers will there be, now and in two years?
- ☐ Is replay or history a requirement?
- ☐ What order must hold, and per which key?
- ☐ Is every handler idempotent and every call bounded?
- ☐ How will the contract be versioned?

Vivekanand Pujar | @VivekanandPujar

SESSION 07 · EVENT-BASED PATTERNS

Four event patterns, one question: what travels in the message?

01 EVENT NOTIFICATION

Something happened.

A thin message says so. Receivers call back for the detail.

```
OrderPlaced
{ id: 42 }
```

02 STATE TRANSFER

Here is the new state.

The message carries the data, so receivers keep a copy of their own.

```
AddressChanged
{ id: 7, address: {…} }
```

03 EVENT SOURCING

The events are the record.

State is not stored; it is derived by replaying the events.

```
Deposited +100
Withdrew −30
```

04 CQRS

Writing and reading part company.

One model takes commands; others, built from events, answer queries.

```
PlaceOrder → OrderPlaced
→ order-history view
```

> **They combine.** The four are independent. Many systems use the first two for years and never need the last two.

Classification after Martin Fowler, “What do you mean by ‘Event-Driven’?”, martinfowler.com, 2017.

Vivekanand Pujar | @VivekanandPujar

SESSION 07 · EVENT-BASED PATTERNS

A notification says that something happened; state transfer says what it now looks like

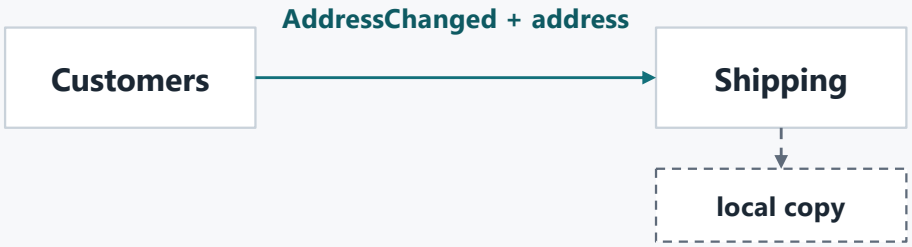
EVENT NOTIFICATION · A THIN MESSAGE



Use it when the receiver needs the latest truth and can afford to ask for it.

The bill. A burst of events becomes a burst of callbacks, and the sender must be up to answer.

EVENT-CARRIED STATE TRANSFER · A FAT MESSAGE



Use it when receivers must keep working while the sender is down.

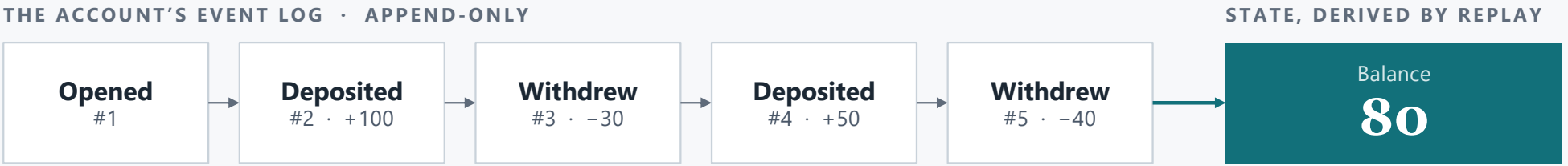
The bill. Duplicated data, stale copies, and a privacy question that every copy must answer.

> **In practice.** Shipping needs an address. With a notification it asks Customers every time. With state transfer it already has one, perhaps a day out of date.

Vivekanand Pujar | @VivekanandPujar

SESSION 07 · EVENT-BASED PATTERNS

Event sourcing stores the history and derives the present from it



Snapshot after #3: balance 70, so a replay can start at #4.

What it gives you

A complete audit trail, and the power to ask what the balance was on 1 March.

What it asks of you

Versioned event schemas, a plan for deletion requests such as crypto-shredding, and snapshots.

When to use it

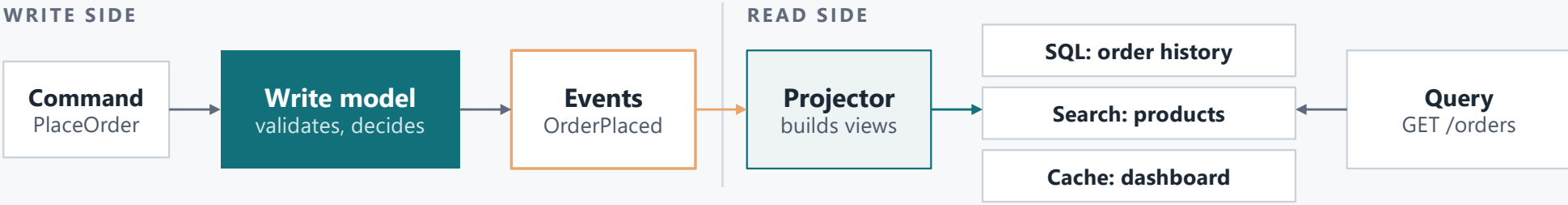
When history is the product: ledgers, orders, compliance. Not for a settings page.

> **The test.** If someone will ask how you got here, store the journey, not just the destination.

Vivekanand Pujar | @VivekanandPujar

SESSION 07 · EVENT-BASED PATTERNS

CQRS lets the model that decides differ from the model that answers



Why split them

Reads outnumber writes many times over. Each side is tuned and scaled on its own.

The price

The read side is always slightly in the past, like starlight. Design the screen for that.

When to say no

Plain CRUD at modest load. CQRS answers a question that many systems never ask.

> **In practice.** A retailer takes orders on a small, strict write model and serves search, history and dashboards from three views built from the same events.

Illustrative scenario.

Vivekanand Pujar | @VivekanandPujar

SESSION 08 · CASE STUDY 5: RABBITMQ AND KAFKA

RabbitMQ delivers and forgets; Kafka remembers and lets readers return

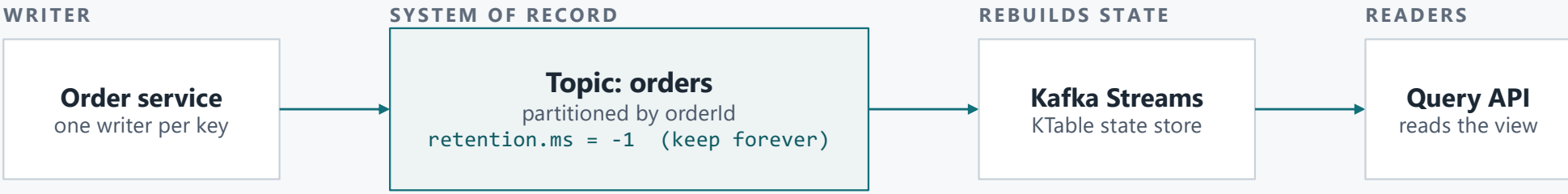
ASPECT	RABBITMQ	APACHE KAFKA
Model	A smart broker: exchanges route messages to queues by rule.	A simple broker and smart consumers: a partitioned, append-only log.
After delivery	The message is deleted once a consumer acknowledges it.	The message stays for a set time or size; each reader keeps its own offset.
Replay	Not by default; RabbitMQ Streams add it.	Native: move the offset and read again.
Ordering	Per queue, weakened by competing consumers.	Per partition, and so per key.
Best at	Task queues, routing, request and reply.	Event streams, event sourcing, change data capture, many readers.

> **A heuristic.** If a message is a job to be done once, use a queue. If it is a fact for many readers, for as long as it matters, use a log.

Vivekanand Pujar | @VivekanandPujar

SESSION 08 · CASE STUDY 6: KAFKA AS AN EVENT STORE

Kafka can be the system of record, if you supply the concurrency check yourself



WHAT KAFKA GIVES YOU

Order per key. Events for one aggregate stay in sequence.

Retention you choose. Including forever, at the price of disk.

Replay. Reset an offset and rebuild any view.

Compaction. Keeps the latest record per key, for snapshots.

WHAT YOU MUST SUPPLY

A concurrency check. Kafka cannot append only if the version is N; route each key to one writer.

Schema discipline. A schema registry, and upcasting of old events.

A read model. Kafka is a log, not a query engine.

A deletion plan. Retained events meet privacy law; encrypt per subject.

Vivekanand Pujar | @VivekanandPujar

CLOSING

Six questions to carry into the training

01 Which of your services carry cross-cutting code that a sidecar or gateway could own?

02 Where does a legacy model leak into your domain code today?

03 Which calls cross a balancer, and does that balancer need to read the request?

04 Where does a callback fail the moment the sender is down?

05 Which of your data would you rather replay than restore?

06 Which queues should be logs, and which logs should be queues?

Vivekanand Pujar | @VivekanandPujar